

SQL Part 3: Joins Explained Simply (INNER JOIN, LEFT JOIN & Keys) — Free Practice Guide

Beyond School · Ready to learn in a fun way! Anyone can learn — easy! Nothing to install, just a web browser.

Step 1: Open a free online SQL playground

- sqliteonline.com — choose "SQLite", paste, click Run.
- db-fiddle.com — pick "SQLite" as the database, paste and Run.
- sqlime.org — a simple SQLite editor in your browser.
- sqlbolt.com — free interactive lessons for extra practice.

Step 2: Paste this setup script and click Run

It creates the same library tables used in the video. If anything goes wrong, just run it again in a fresh playground.

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  title TEXT,
  author TEXT,
  year INTEGER,
  genre TEXT,
  pages INTEGER
);
INSERT INTO books VALUES
  (1, 'Harry Potter', 'J.K. Rowling', 1997, 'Fantasy', 309),
  (2, 'Matilda', 'Roald Dahl', 1988, 'Comedy', 240),
  (3, 'Wonder', 'R.J. Palacio', 2012, 'Drama', 310),
  (4, 'The Hobbit', 'J.R.R. Tolkien', 1937, 'Fantasy', 310),
  (5, 'Holes', 'Louis Sachar', 1998, 'Mystery', 233),
  (6, 'Charlotte''s Web', 'E.B. White', 1952, 'Classic', 192),
  (7, 'Dog Man', 'Dav Pilkey', 2016, 'Comedy', 240),
  (8, 'Coraline', 'Neil Gaiman', 2002, 'Fantasy', 162);

CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  name TEXT,
  city TEXT,
  joined TEXT,
```

```
  email TEXT
);
INSERT INTO members VALUES
  (1, 'Aisha', 'Austin', '2023-01-15', 'aisha@mail.com'),
  (2, 'Ben', 'Denver', '2023-03-02', NULL),
  (3, 'Carlos', 'Austin', '2024-06-20', 'carlos@mail.com'),
  (4, 'Dana', 'Boston', '2024-09-10', NULL),
  (5, 'Eli', 'Denver', '2025-02-01', 'eli@mail.com');

CREATE TABLE loans (
  loan_id INTEGER PRIMARY KEY,
  book_id INTEGER,
  member_id INTEGER,
  loan_date TEXT,
  return_date TEXT
);
INSERT INTO loans VALUES
  (1, 1, 1, '2026-01-05', '2026-01-19'),
  (2, 3, 1, '2026-02-10', NULL),
  (3, 2, 2, '2026-01-20', '2026-02-03'),
  (4, 4, 3, '2026-03-01', '2026-03-15'),
  (5, 1, 3, '2026-04-02', NULL),
  (6, 7, 4, '2026-04-10', '2026-04-24'),
  (7, 7, 2, '2026-05-01', NULL),
  (8, 5, 4, '2026-05-15', NULL);
```

Step 3: Run every query from the video, in order

Paste one at a time, click Run, and compare with the expected result.

The members table

```
SELECT * FROM members;
```

```
Expected result:
member_id | name | city | joined | email
1 | Aisha | Austin | 2023-01-15 | aisha@mail.com
2 | Ben | Denver | 2023-03-02 | NULL
3 | Carlos | Austin | 2024-06-20 | carlos@mail.com
4 | Dana | Boston | 2024-09-10 | NULL
5 | Eli | Denver | 2025-02-01 | eli@mail.com
```

The loans table

```
SELECT * FROM loans;
```

```
Expected result:
loan_id | book_id | member_id | loan_date | return_date
1 | 1 | 1 | 2026-01-05 | 2026-01-19
2 | 3 | 1 | 2026-02-10 | NULL
3 | 2 | 2 | 2026-01-20 | 2026-02-03
4 | 4 | 3 | 2026-03-01 | 2026-03-15
5 | 1 | 3 | 2026-04-02 | NULL
6 | 7 | 4 | 2026-04-10 | 2026-04-24
7 | 7 | 2 | 2026-05-01 | NULL
8 | 5 | 4 | 2026-05-15 | NULL
```

INNER JOIN

```
SELECT members.name, loans.book_id,          loans.loan_date FROM loans INNER JOIN members    ON
loans.member_id = members.member_id;
```

```
Expected result:
name | book_id | loan_date
Aisha | 1 | 2026-01-05
Aisha | 3 | 2026-02-10
```

```
Ben | 2 | 2026-01-20
Carlos | 4 | 2026-03-01
Carlos | 1 | 2026-04-02
Dana | 7 | 2026-04-10
Ben | 7 | 2026-05-01
Dana | 5 | 2026-05-15
```

Short table nicknames

```
SELECT m.name, l.loan_date FROM loans AS l JOIN members AS m   ON l.member_id = m.member_id WHERE m.city = 'Austin';
```

```
Expected result:
name | loan_date
Aisha | 2026-01-05
Aisha | 2026-02-10
Carlos | 2026-03-01
Carlos | 2026-04-02
```

Joining three tables

```
SELECT m.name, b.title, l.loan_date FROM loans AS l JOIN members AS m ON l.member_id = m.member_id JOIN books AS b ON l.book_id = b.id ORDER BY l.loan_date;
```

```
Expected result:
name | title | loan_date
Aisha | Harry Potter | 2026-01-05
Ben | Matilda | 2026-01-20
Aisha | Wonder | 2026-02-10
Carlos | The Hobbit | 2026-03-01
Carlos | Harry Potter | 2026-04-02
Dana | Dog Man | 2026-04-10
Ben | Dog Man | 2026-05-01
Dana | Holes | 2026-05-15
```

LEFT JOIN

```
SELECT m.name, l.loan_id, l.book_id FROM members AS m LEFT JOIN loans AS l    ON m.member_id = l.member_id;
```

```
Expected result:
name | loan_id | book_id
Aisha | 1 | 1
Aisha | 2 | 3
Ben | 3 | 2
Ben | 7 | 7
Carlos | 5 | 1
Carlos | 4 | 4
Dana | 8 | 5
Dana | 6 | 7
Eli | NULL | NULL
```

Finding unmatched rows

```
SELECT m.name FROM members AS m LEFT JOIN loans AS l    ON m.member_id = l.member_id WHERE l.loan_id IS NULL;
```

```
Expected result:
name
Eli
```

Books never borrowed

```
SELECT b.title FROM books AS b LEFT JOIN loans AS l ON b.id = l.book_id WHERE l.loan_id IS NULL;
```

```
Expected result:
title
Charlotte's Web
Coraline
```

FULL JOIN example

```
SELECT m.name, l.loan_id FROM members AS m FULL JOIN loans AS l   ON m.member_id = l.member_id ORDER BY l.loan_id;
```

```
Expected result:
name | loan_id
Eli | NULL
Aisha | 1
Aisha | 2
Ben | 3
Carlos | 4
Carlos | 5
Dana | 6
Ben | 7
Dana | 8
```

RIGHT JOIN example

```
SELECT l.loan_id, m.name FROM members AS m RIGHT JOIN loans AS l   ON m.member_id = l.member_id;
```

```
Expected result:
loan_id | name
1 | Aisha
2 | Aisha
3 | Ben
7 | Ben
4 | Carlos
5 | Carlos
6 | Dana
8 | Dana
```

JOIN + GROUP BY

```
SELECT m.name, COUNT(l.loan_id) AS loans FROM members AS m LEFT JOIN loans AS l   ON m.member_id = l.member_id GROUP BY m.name ORDER BY loans DESC;
```

```
Expected result:
name | loans
Dana | 2
Carlos | 2
Ben | 2
Aisha | 2
Eli | 0
```

JOIN + WHERE

```
SELECT m.name, b.title, l.loan_date FROM loans AS l JOIN members AS m ON l.member_id = m.member_id JOIN books AS b ON l.book_id = b.id WHERE l.return_date IS NULL;
```

```
Expected result:
name | title | loan_date
Aisha | Wonder | 2026-02-10
Carlos | Harry Potter | 2026-04-02
Ben | Dog Man | 2026-05-01
Dana | Holes | 2026-05-15
```

JOIN + GROUP BY city

```
SELECT m.city, COUNT(*) AS loans FROM loans AS l JOIN members AS m ON l.member_id = m.member_id GROUP BY m.city ORDER BY loans DESC;
```

```
Expected result:
city | loans
Austin | 4
Denver | 2
Boston | 2
```

Most popular book

```
SELECT b.title, COUNT(*) AS times FROM loans AS l JOIN books AS b ON l.book_id = b.id GROUP BY b.title ORDER BY times DESC LIMIT 2;
```

```
Expected result:
title | times
Harry Potter | 2
Dog Man | 2
```

Step 4: Challenges (try first!)

C1. List every book title with how many times it was borrowed (include zeros).
C2. Show the titles Aisha borrowed.
C3. Which books are still out (not returned)? Show title and member.

Answers

C1. List every book title with how many times it was borrowed (include zeros).

```
SELECT b.title, COUNT(l.loan_id) AS times_borrowed
FROM books AS b LEFT JOIN loans AS l ON b.id = l.book_id
GROUP BY b.title ORDER BY times_borrowed DESC;
```

```
Expected result:
title | times_borrowed
Harry Potter | 2
Dog Man | 2
Wonder | 1
The Hobbit | 1
Matilda | 1
Holes | 1
Coraline | 0
Charlotte's Web | 0
```

C2. Show the titles Aisha borrowed.

```
SELECT b.title FROM loans AS l
JOIN members AS m ON l.member_id = m.member_id
JOIN books AS b ON l.book_id = b.id
WHERE m.name = 'Aisha';
```

```
Expected result:
title
Harry Potter
Wonder
```

C3. Which books are still out (not returned)? Show title and member.

```
SELECT b.title, m.name FROM loans AS l
JOIN books AS b ON l.book_id = b.id
JOIN members AS m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

```
Expected result:
title | name
Wonder | Aisha
Harry Potter | Carlos
Dog Man | Ben
Holes | Dana
```

Keep going!

Watch the whole free SQL course on Beyond School, starting with SQL Part 1: https://youtu.be/72JvYu2OtiA
Beyond School — Ready to learn in a fun way!